

# Transformers

miércoles, 22 de abril de 2026 03:12 p. m.

From LLMs, we know:

- Language modeling = predicting the next token
- MLE / cross-entropy objective
- Neural LMs and contextual representations (ELMo)
- Training on massive text corpora

But we still have problems:

- Sequential models are slow
- Long-range dependencies are hard
- Context building is indirect

**Transformers solve these problems by changing how context is computed.**

## Self-Attention

### Intuition

Self-attention lets each word:

- Look at **all other words**
- Decide which ones matter
- Combine their information

Instead of:

- Compressing everything into one hidden state
- Passing information step-by-step

### One-minute mental model

For each word:

1. Ask: *What am I looking for?*
2. Compare with all other words
3. Take a weighted average of what matters

Key consequence:

- Any word can directly influence any other word
- Long-distance dependencies are easy

## Transformer Blocks (Architecture Overview)

**One Transformer layer consists of:**

1. Self-attention
2. Add & LayerNorm
3. Feed-forward network
4. Add & LayerNorm

## Feed-forward network (important reminder)

Applied independently to each position:

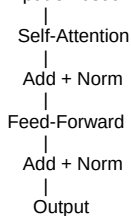
$$FFN(\mathbf{x}) = \max(0, W_1 \mathbf{x} + b_1) W_2 + b_2$$

Interpretation:

- Adds nonlinearity
- Increases expressive power
- Same weights for all positions

## Architecture sketch (board-friendly)

Input embeddings



Stack many such layers → **Transformer**

### Why the name “Transformer”?

The name reflects what the model does conceptually:

- It **transforms** a sequence of input representations
- Into a sequence of **contextualized representations**
- Using attention-based transformations at each layer

## Sampling in LLMs and Transformers

### 1. What is sampling?

At a high level, **sampling** is the procedure used during **generation** to turn a model's *predicted probability distribution* into an actual next token.

A language model—whether it is small or large, RNN-based or Transformer-based—defines the same quantity:

$$P(w_t \mid w_1^{t-1})$$

That is:

**Given the tokens generated so far, what is the probability of each possible next token?**

Sampling answers the follow-up question:

**Given this distribution, which token do we actually choose?**

## 2. Sampling is independent of architecture

A crucial conceptual point:

**Sampling does not depend on whether the model is an LLM or a Transformer.**

- **Transformers** specify *how the probability distribution is computed*
- **Sampling** specifies *how we select a token from that distribution*

This separation is important pedagogically.

- A Transformer computes scores (logits) using self-attention
- An RNN computes scores using a recurrent hidden state
- Once scores are produced, **sampling works the same way**

## 3. Unified generation loop (LLMs + Transformers)

The generation process is **autoregressive** for *all* standard LLMs:

1. Take the tokens generated so far:  $w_1^{t-1}$
2. Compute scores (logits) for all tokens in the vocabulary
3. Apply **softmax** to get probabilities
4. **Sample** one token using a decoding strategy
5. Append the token and repeat

Important clarification (often misunderstood by students):

**Transformers are parallel during training, but sequential during sampling.**

Sampling must be sequential because each token conditions on the previous one.

## 4. Sampling strategies (model-agnostic)

Once we have a probability distribution, we can apply different strategies:

### 4.1 Greedy decoding

- Always choose the most probable token
- Deterministic
- Often repetitive and dull

### 4.2 Multinomial sampling

- Sample proportional to probabilities
- More diverse
- Risk of incoherence if low-probability tokens are sampled too often

### 4.3 Temperature scaling

$$P_T(w) \propto \exp\left(\frac{S_w}{T}\right)$$

- Low temperature  $\rightarrow$  conservative, predictable
- High temperature  $\rightarrow$  creative, risky

### 4.4 Top-k and top-p (nucleus) sampling

- Restrict sampling to a subset of likely tokens
- Balance coherence and diversity

**Key idea:**

These strategies affect *style and variability*, not the underlying model's knowledge.

## 5. What changed with Transformers (and what didn't)

What **did not change**:

- The probability objective
- The autoregressive generation loop
- The need for sampling

What **did change**:

- The *quality* of the probability distribution
- The ability to model long-range dependencies
- The consistency of generated text

Thus:

**Transformers don't change how we sample—only what we sample from.**

## Limitations of LLMs and Transformers

Despite their success, LLMs (including Transformer-based ones) have **fundamental limitations** that are important to understand.

### 1. Lack of true reasoning and understanding

LLMs:

- Predict the next token using statistical regularities
- Do not manipulate symbols using explicit rules
- Do not "execute" logical reasoning in the traditional sense

As a result:

- They may produce plausible but incorrect explanations
- Errors often surface in long multi-step reasoning

### 2. Arithmetic and exact computation

A well-known limitation concerns **arithmetic operations**.

Example:

- Long addition, multiplication of large integers
- Precise multi-step calculations

Why this happens:

- LLMs learn arithmetic as **patterns in text**, not as algorithms
- Training optimizes likelihood, not numerical correctness
- Results degrade as numbers grow or formatting changes

This explains why:

- Small arithmetic may be correct
- Slightly different phrasing can cause failure

**Transformers do not natively implement arithmetic algorithms.**

(External tools or symbolic modules are required for reliability.)

### 3. Sensitivity to prompting and sampling

Due to sampling:

- Small changes in temperature or prompt can produce very different outputs
- The model may contradict itself across generations

This is not randomness in the model's knowledge, but in:

- How probability mass is explored during sampling

### 4. Context and generalization limits

Even with self-attention:

- Models have finite context windows
- Information outside the window is inaccessible
- Internal "memory" is implicit and brittle

Additionally:

- Generalization to rare or adversarial inputs is weak
- Training data biases are reflected in outputs